Executive Summary for Group 3, Sea Turtle On Land.

Calvin Hargis, Shaotong Hong, Kitili Kisinguh, Shahzeb Pasha.

Task outline.

In this task, much like the previous assignment, the team was to make a robot capable of generating a map of the environment for navigation, navigating around that map autonomously, visually detecting a specified target, and navigating to that target. All of this was to take place within a simulated Gazebo environment designed to emulate different use-case scenarios and test Shelly's limits. The team decided that this could be a model for using a biometric robot that appears to be a natural animal in order to remotely monitor and collect data on a protected or sensitive ecosystem. In this way, Shelly could act as the eyes of wildlife researchers who cannot enter the area they're studying, while shelly navigates autonomously and collects data for them to review later. This project uses this idea to focus the discussion around Shelly's limitations and achievements, but is not attempting to achieve this currently, merely acting as a model or demonstration.

Navigation.

In this paper, Shelly is able to navigate using the ROS2 NAV2 stack. This allows the robot to use a Global cost map, which evaluates the optimal paths at all times given the total known area, and the Local cost map, which centres around the robot and allows it to process dynamic changes in the immediate environment. This is accomplished with the use of plugins which are tailored to the specific needs of this project. This includes enabling and utilizing the rolling window, collision monitor, AMCL-Adaptive Monte Carlo Localization, and twist mux. The result is that Shelly is able to reliably and confidently navigate around an unknown environment and stare acquired data for later. However, there are limitations regarding the cohesion between Gazebo and RVIS and also some phantom obstacles that appear.

Mapping.

In order to engage odometry for mapping, Shelly was given wheels underneath her flippers, allowing differential drive. The mapping makes use of the gz_bridge node and the slam_toolbox node to exchange LiDAR and odometry data. By using these available tools, a continuously updated occupancy grid is generated that can tell the aforementioned NAV2 stack where Shelly can and cannot enter. The current mapping system can weather inconsistent data rates while also allowing asynchronous processing, higher efficiency, and improved responsiveness. Trials in the different designed worlds demonstrate success and consistency in low-detailed worlds but significant hardware challenged in high-detailed ones. When running in these worlds the four

core CPU usage is nearly 100%.

Target Detection.

The target detection program is currently in-between two different design models and so does not produce a workable output. For this reason, this paper focuses on comparison between the two developed models. The first, Array Analysis, focuses on pixel location data and statistical analysis to filter out unwanted detected pixels. However, this model was delicate due to the many changes in data type, and also computationally expensive leading to crashes. The second model is Image Analysis, predicated on complex image processing techniques with an emphasis on data-type consistency and reducing code size. This model took less time to develop, is faster, and is smaller, however it does require the user to know many image processing techniques beyond the scope of ELEC330. This model's main limitation is that it was never completed due to time constraints.

World Design.

In this paper is a detailed discussion of the different worlds completed for the purpose of testing the robot's abilities. There are four such worlds: One designed to closely mimic a beach, one designed with less harsh slopes and fewer obstacles, one designed as an obstacle course with narrow passages, and one designed with complex ground structures to mimic a maze. Each of these was developed in blender and exported to Gazebo to push and pull at the limitations of specific aspects of Shelly's design.

Conclusion.

With reference to modelling the aforementioned environmental surveillance robot, Shelly, in her current form, can be seen as a good proof of concept, but one that needs further attention. Navigation functions reliably and fully utilizes the NAV2 stack, allowing Shelly to run autonomously with no direction. However, the data processing results in phantom obstacles telling the robot not to move in a space that is completely passable, something that would inhibit functionality in the wild. The mapping provides useful and detailed occupancy grids using a single sensor, enable that NAV2 movement and constantly updating it. However, the mapping also runs into issues with CPU usage and data processing that results in a slow and sometimes error-prone outcomes when used in worlds with higher detail, such as the real world. The target detection has previously been show to work with still images but required a complete and, so far, un-finished redesign to handle the higher data flow of video. The common denominator for the problems faced by Shelly are limitations around data processing, which can be improved upon in a future model by upgrading the hardware beyond the current four cores and developing slimmer and more efficient programs to complete each task.

1. Introduction.

As stated in the executive summary, Shelly is designed to be a model for autonomous animal surveillance of protected areas performed by a biometric robot. The idea is that Shelly will be able to navigate through an environment disguised as a sea turtle on land and catalogue what she sees. Specifically, she will use image processing to detect new sea-turtles and track population data. To enable this, the team designed a navigation system based on the ROS2 NAV2 stack, mapping based on LiDAR scanning, and tested these in simulated Gazebo worlds which present different levels of challenge and complexity. It should be stated that this is not a project that could be quickly turned into a working robot immediately, but it hopes to act as a guiding demonstration of how such a system might work in the future.

This detailed section of the paper will walk through the project by talking about the four major components of its development: Navigation, Mapping, Target Detection, and World Design. Each of these sub-sections will thoroughly explain the design process and limitations and then discuss what future and further improvements could be made.

2. Contribution.

2.1 Kisinguh.

Kitili Kisinguh was responsible for the implementation of the robots Navigation stack and the integration of sensor fusion within the autonomous system. This involved configuring and deploying the ROS2 NAV2 stack to enable the robot to plan and follow path in a dynamic environment using real time map data and obstacle avoidance strategies. He set up key components such as global and local costmaps, behavior trees for task execution and navigation planners to ensure accurate goal execution and responsive path correction. In addition, he implemented sensor fusion using tools such as the Extended Kalman Filter (EKF) to combine data from multiple sensors such as the wheel odometry, Lidar and the IMU to produce a reliable pose estimate. Kitili’s work enabled the robot to perform robust and intelligent autonomous movement, seamlessly integrating mapping planning and localization.

2.2 Hong.

Shaotong Hong was primarily responsible for the design and implementation of the SLAM

subsystem on the turtle robot, a critical component enabling autonomous navigation. His work ensured the real-time generation of an occupancy grid map alongside a map → odom transform, both of which are essential for seamless integration with the Nav2 navigation framework. In Assignment 3, he transitioned the system architecture from the computationally intensive 3D LiDAR-based SLAM approach (used in Assignment 2 via the lidarslam package) to a more efficient 2D laser-based solution using slam_toolbox in online_async mode. This change not only reduced system load—allowing for real-time performance on resource-constrained hardware—but also improved compatibility with the required odometry-based TF hierarchy.

In addition to his core responsibilities, Shaotong implemented a backup target-recognition pipeline using Gazebo's bounding box camera plugin. Although the primary object recognition task, based on OpenCV, was handled by another team member, this supplementary method provided a reliable contingency pathway for validating the autonomous target navigation behavior in simulation.

2.3 Hargis.

Hargis' main contributions were target detection and group leadership. The target detection was a task that Hargis took on independently, trialing two methods based on different principles of data manipulation. The first was designed around manipulating the pixel location data and using statistical analysis based on normal distribution to isolate only the high utility detected pixels. This method eventually became too cumbersome, computationally expensive, and time consuming to implement and so the second method was developed. The second method relied on using image processing techniques to filter the usable data while keeping continuity in data types. This was much faster in implementation, but unfortunately not fast enough to be finished before the deadline. Both of these methods have evidence pulled from previous works of Hargis and this team to compare and contrast potential use cases for the future.

In terms of leadership, Hargis was tasked with setting internal deadlines, meeting dates, and standards. Hargis was responsible for internal cohesion, usually calling and checking in on the progress of other members of the team on a weekly basis. Furthermore, he was responsible for communicating between the team and the department at the University of Liverpool should any such questions arise. When the bench inspection and the final report were being prepared, it was Hargis who put together the structure, led the practice sessions, and cohesively combined the entire report.

2.4 Pasha.

Through this project, Shahzeb developed a lot of technical expertise in creating the simulated beach environment. One of the key areas of improvement was 3D modelling with Blender, where Shahzeb learned techniques for terrain sculpting, mesh optimisation and procedural texturing through online tutorials, software documentation and trial by hands. Learning to master tools like dynamic topology and decimation modifiers became essential to achieve the ideal balance between simulation performance and visual quality. Shahzeb also gained valuable experience in cross-platform integration, addressing problems such as coordinate system differences and texture compatibility between Blender and Gazebo. This included reading through ROS and Gazebo documentation, technical forums and repeated testing to verify correct model behavior within the physics simulation.

3. Navigation.

Robot Navigation is structured around a modular framework, with the Nav2 (Navigation 2) stack serving as the core component. Nav2 facilitates autonomous navigation in complex dynamic environment and execution of user specified goals. It integrates a static map form the SLAM toolbox with real time sensor data to perform global and local path planning, velocity command generation, and obstacle avoidance.

As depicted in Figure 1, the NAV2 architecture consists of several key subsystems, central to its operation is the behavior tree, “bt_navigator”, which performs high level task execution through a hierarchal and modular structure. This allows for flexible decision making and robust handling of navigation goals, Nav2 also includes an array of optimized servers responsible for specific functions such as path planning’ “planner_server”, behavior control; “bt_navigator”, and motion control “controller_server”. These components communicate via ROS2 nodes to provide reliable navigation performance.

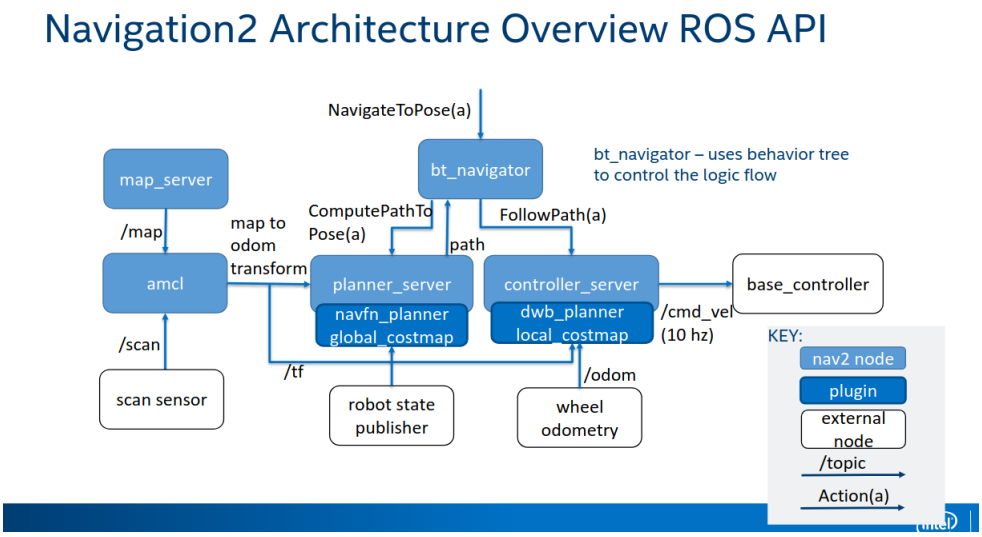

Figure 1. Navigation2 Architecture [1]

The inputs to the NAV2 framework are tf transforms which correspond to the REP-105 ROS standard, a map source provided by the slam_toolbox as well as other relevant sensor data, ie the lidar data, the corresponding output from the framework will be the appropriate velocity commands for the various motors.

3.1 Navigation Parameters.

Besides installing the navigation package and loading an appropriate map, Nav2'S behavior is governed by the parameters set in the nav2_params.yaml file, where all the navigation stacks functionalities are configured, with the key parameter of focus being the cost map.

A cost map is a grid-based representation in which each cell is assigned a numerical cost reflecting the difficulty for a robot to transverse it, the planning server from the behavior tree would rather the robot transverse areas assigned with low-cost cells and avoid areas with high-cost cells. The cost map is derived from the underlying map and is continuously updated using LiDAR and other sensor inputs to produce an accurate 2D or 3D occupancy model.

In NaV2, overall navigation is performed through modules such as the "bt_navigator" discussed above, the waypoint _follower, "controller_server", "planner_server" and "recoveries_server", the waypoint follower manages navigation through predefined goals with minimal configuration, while the recoveries server executes fallback behaviors such as spin, backup or wait when the navigation fails.

In the ROS2 navigation stack, two distinct cosmaps operate cohesively to model obstacles and guide motion planning:

- **Global Cost Map**: Covers the entire known environment and is used by the global planner to compute optimal paths between two robot poses. It uses a preloaded static map from the map server and overlays dynamic obstacles detected by sensors onto this map, enabling a cost value to be assigned to each grid cell, representing the difficulty of traversal.
- **Local Cost Map**: Focused on short term planning, the local cost map maintains rolling, robot centered window that continuously moves with the robot, as it is constantly updated by real time live sensor data, it is well suited for dynamic obstacle detection and avoidance.

The Global and Local costmaps of the framework are updated with the /scan2 topic produced from the LiDAR, specified in the .yaml file the overall resolution per cell for both cost maps is set to 0.05m, balancing spatial accuracy and computational load. The local cost map is configured with a 3x3 meter area and has a 5Hz update frequency, whereas the global costmap updates every 1Hz, ensuring responsive obstacle tracking for short motion planning. Aside from the features provided by the Global and Local costmaps the ObstacleLayer and InflationLayer are key plugins that complete the framework providing further functionality to enhance real time obstacle avoidance. The Obstaclelayer uses the data from the /scan2 topic to detect and mark obstacles in real time, the InflationLayer expands obstacle cost into surrounding cells using a

decay curve defined by the inflation radius of 0.7m and cost scaling factor of 3.0, which creates a buffer zone that encourages the planner and controller to choose safer paths.

The rolling window behavior in the local costmap is controlled by the parameter rolling window: true. When enabled, this ensures that he local costmap continuously shifts its origin to remain centered around the robot's current position as it moves. This allows the local planner to always have an up to date view of the immediate surroundings.

The final key parameter is the collision monitor, a dedicated module in the Nav2 stack designed to enhance real time safety by predicting and reacting to imminent collisions, configured under the collision monitor namespace it uses data from the lidar /scan2 topic to monitor the robot's immediate surroundings. It defines action zones around the robot using configurable polygons which determine how the robot should respond when potential collisions are detected. The monitor operates with a simulation time step of 0.1 second and uses parameters such as min height, max height and time_before_collision to filter valid obstacle data and predict collisions in advance, depending on the severity and proximity of detected obstacles, it can issue commands to stop, slow down or limit the robots movement, ensuring safer operation in clutters or unpredictable environments.

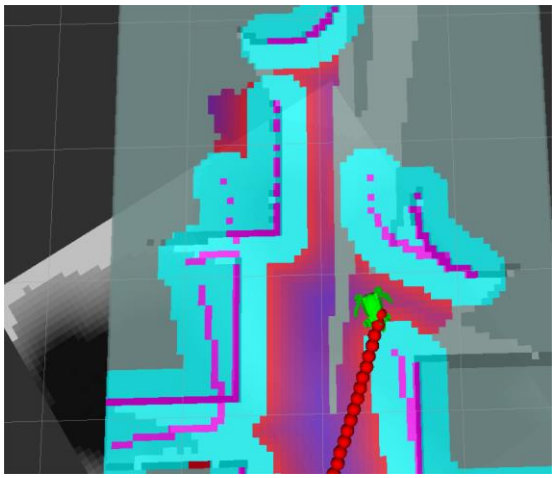

Figure 2. Global and Local Costmap

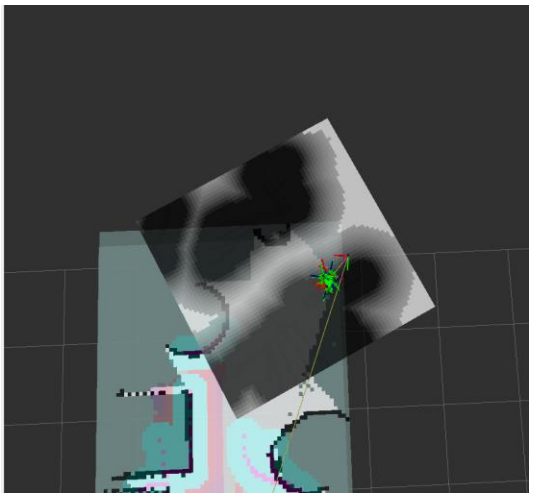

Figure 3. Local Costmap Rolling Window

3.2 AMCL-Adaptive Monte Carlo Localization.

An inherit component of the ROS navigation stack is the use of AMCL (Adaptive Monte Carlo Localization), an adaptive particle filter based probabilistic algorithm for 2D localization, the AMCL algorithm attempts to estimate the robots pose on a predefined static map by fusing a static map provided by the map server and incoming sensor data; typically from a 2D LiDAR to provide a statistical estimate of the robots pose on the map in relation to the simulation environment; Gazebo. To maintain localization accuracy, ACML manages a dynamic set of weighted particles, each representing a potential robot pose. These particles are continuously updated and resampled based on how well their predicted sensor observations match the actual incoming data ensuring the pose estimate reflects the most probable location of the robot within the map. The algorithm output is a transform between the map and odom frames, allowing the robots localized pose to be continuously updated and shared across the navigation stack.

3.3 Twist Mux.

In the robot's control architecture, the twist_mux package serves as an additional command layer, responsible for multiplexing multiple velocity command sources and forwarding the highest priority command to the robot's actuators via the “cmd_vel_out” topic. It operates on “geometry_msgs/Twist” messages and ensures exclusive control by allowing only one input source to publish velocity commands at any given time, this layer is governed by a .yaml file that defines each input source, priority level and associated timeout. If a higher priority input becomes inactive, i.e. fails to publish within its timeout window, control is automatically passed to the next available source in the priority hierarchy, enabling robust and flexible command management.

In the twist_mux.yaml configuration two input topics are defined

- /cmd_vel_target, velocity messages from the bounding box package, it has the highest priority of 100 and enables the robot to navigate towards a labeled objects if detected by the camera.
- /cmd_vel_smoothed, velocity messages from the Nav2 stack, it has a priority of 10 and is responsible for navigating the robot in the simulated environment.

3.4 Automatic Exploration.

Navigation to unexplored areas of the map is achieved using the auto_explore.py script, a custom autonomous exploration node within the NAV2 stack, its primary function is to systematically explore unidentified regions generated by the SLAM toolbox. The node subscribes to the /map topic to receive the occupancy grid and uses NumPy to process the map and detect potential unexplored cells, once a valid frontier is found, a navigation goal is sent to the NAV2 stack via the NavigateToPose action interface. The node operates on a timer, periodically scanning for new frontiers and maintains a set of previously visited frontiers to prevent redundant exploration.

3.5 Discussion.

The performance of the NAV2 framework in the simulated environment shown in Figure 4, demonstrates the effective and reliable autonomous navigation. The robot is spawned in a simulated environment under the namespace “fws_robot_world3.sdf”, which is maze like course with a labeled object at the end for object detection, as seen in the image, the robot successfully follows a planned trajectory represented by the red path in RVIZ and is able to park next to the designated object.

This behavior is made possible through the integration of the systems discussed above, the robot relies on data from the LiDAR sensor, published to the /scan2 topic and processed in real time by the local and global costmaps. The rolling local costmap as illustrated in Figure 3 continuously shifts its origin in relation to the center of the robot provides a dynamic view of the immediate surroundings, allowing the path planner and controller to respond to nearby obstacles. The inflation layer of the costmap, visible in the Rviz panel as concentric gradients primarily cyan and purple provides a safety buffer by expanding the obstacle cost outwards ensuring the robot steers towards lower cost paths, although not visible the collision monitor uses the aforementioned data to provide the most efficient path to navigate to a pose.

Overall, the integration of sensor fusion, real time costmap updates and behavior tree-based navigation logic enables the robot to autonomously complete its navigation task with a high degree of reliability, although the robot safely navigates the course the overall performance of the navigation stack could be optimized. Analysis of the Rviz visualization reveals that the robot

occasionally intrudes into the inflation layer surrounding obtacles, suggesting that certain NAV2 parameters such as the inflation radius, cost scaling factor or local planner tolerances may require further tuning to ensure more conservative path selection and improved obstacle clearance. Additionally, visual artifacts in the costmap show phantom obstacles, where regions of the grid are marked as occupied despite no corresponding physical object in the Gazebo environment, these discrepancies may stem from LiDAR noise, latency in message synchronization between Gazebo and Rviz, or a mismatch in simulation time and ROS clock.

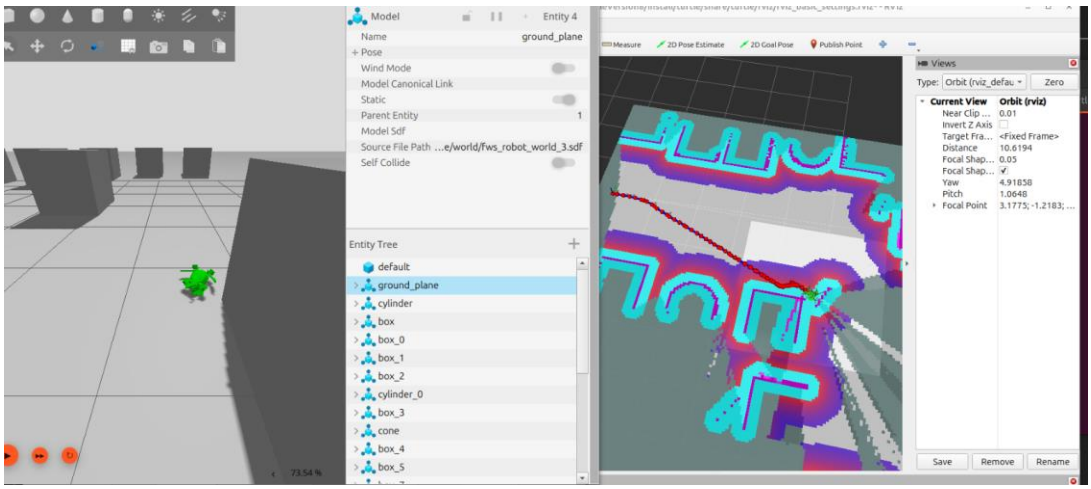

Figure 4. robot Navigation

3.6 EKF Extended Kalman Filter.

As a future enhancement, implementing an Extended Kalman Filter (EKF) node can significantly improve localization accuracy by reducing odometry drift through sensor fusion. In the current setup drift in wheel odometry is currently present and leads to accumulated pose errors that affect both localization and mapping performance. By integrating data from multiple sources such as LiDAR and the inertial measurement unit (IMU), the EKF can provide a more robust and reliable estimate of the robots pose in the map frame. This improved odometry source would likely mitigate the mapping artifacts observed in the costmap.

The EKF functionality is typically implemented using the robot localization package in ROS2, which supports multi-sensor fusion, with the configuration performed in the ekf.yaml file. The file configuration defines the input topics such as /odom, /imu/data and /scan2 and species which sensor fields should be used for the state estimation, i.e position, orientation, linear acceleration and yaw rate. The EKF node then publishes a fused odometry to the topic /odometry/filtered which can replace the raw odometry input for downstream components such as the NAV2 localization and planning stack, with proper tuning of covariance matrices and

update rates the EKF can significantly enhance pose accuracy, leading to more stable path planning, improved map consistency and a reduction in false obstacle detections with the costmaps.

4. Mapping.

4.1 Changing robot joint configuration.

Motivation for Reconfiguring the Turtle Robot.

The initial design of the turtle robot featured four legs, each equipped with two joints, to emulate the terrestrial locomotion of a sea turtle. While this biomimetic approach provided a compelling model for gait simulation, it introduced significant challenges for autonomous navigation. The robot's complex, nonholonomic contact patterns pose a challenge to creating a reliable inverse kinematic model to transform joint motions into robot linear and angular odometry. Consequently, the system lacked a dependable odometry, which is crucial for estimating the robot's position over time.

In the Gazebo simulation environment, this limitation meant that the robot could only publish LaserScan data and a partial transform tree extending from the base_footprint frame (representing the projection of the robot's chassis onto the ground plane) to its child frames. However, effective integration with the slam_toolbox package necessitates LaserScan data, accurate odometry information, and a complete transform hierarchy connecting base_footprint, base_link, and Sensor_link.

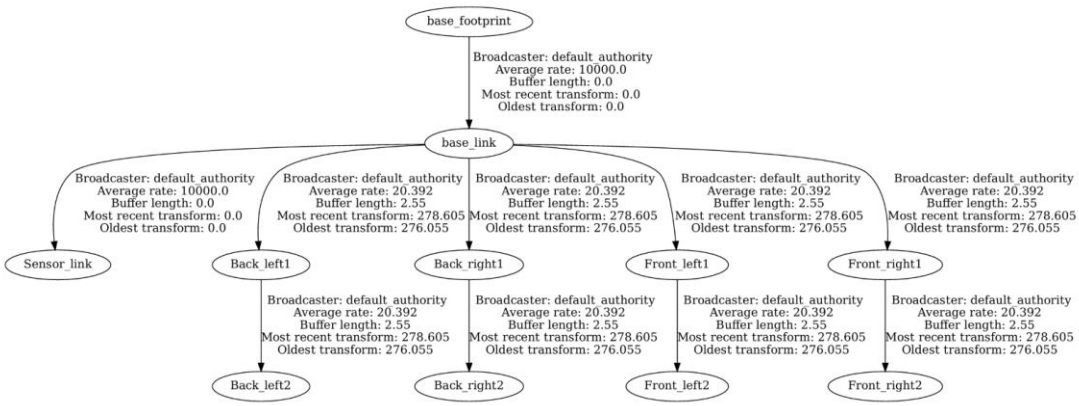

Figure 5: TF tree without odom

To address these challenges and expedite the development of SLAM and target-following navigation functionalities, we consulted with Mr. Ahmed Al-Irhayim and obtained permission to reconfigure the robot from a legged to a wheeled platform. This transition involved adopting a differential drive (diff-drive) configuration, which inherently supports the generation of odometry data and simplifies control mechanisms.

4.2 Implementation of Differential Drive Using Gazebo Plugins.

The differential drive configuration was implemented using Gazebo's built-in DiffDrive plugin. This plugin facilitates the control of robots with two independently driven wheels on either side, allowing for straightforward linear and angular movement. By incorporating the DiffDrive plugin into the robot's URDF (Unified Robot Description Format) file, we enabled the simulation to:

- **Subscribe to velocity commands**: The plugin listens to the /cmd_vel topic, receiving gz.msgs.Twist messages that specify desired linear and angular velocities.
- **Compute and publish odometry**: Based on the wheel velocities, the plugin calculates the robot's pose and publishes gz.msgs.Odometry (bridge to nav_msgs/msg/Odometry)messages on the /odom topic.
- **Broadcast transform frames**: It provides the necessary transform between the odom and base_link frames, completing the transform tree required for SLAM and navigation tasks.

The plugin's configuration includes parameters such as wheel_separation, wheel_radius, and topic names for command and odometry messages. These settings ensure that the simulated robot's behavior closely mirrors that of a physical differential drive robot, facilitating a seamless transition from simulation to real-world deployment.

By reconfiguring the turtle robot to utilize a differential drive system, we established a robust foundation for implementing SLAM and autonomous navigation. This approach not only resolved the limitations posed by the original legged design but also aligned with standard practices in mobile robotics, thereby enhancing the project's scalability and applicability[3].

4.3 Impact of Configuration Change.

A key advantage that enabled this reconfiguration without disrupting the broader software architecture lies in the inherently decoupled nature of the ROS 2 topic-based communication system. In ROS 2, nodes exchange data through named topics in a publisher–subscriber model, which means they do not require direct knowledge of how or where data is generated—only that it conforms to the expected message type and topic name.

Thanks to this modular design, switching from a legged robot to a differential drive configuration only affected the internal method by which odometry (/odom) was generated. All other nodes—such as slam_toolbox and Nav2—continued to operate without modification, as they simply subscribe to the existing topic names and rely on the correct TF and message formats being available. This decoupling greatly streamlined development: we were able to improve system stability and performance by swapping out the locomotion model, without needing to refactor

or rewire downstream SLAM, localisation, or planning components.

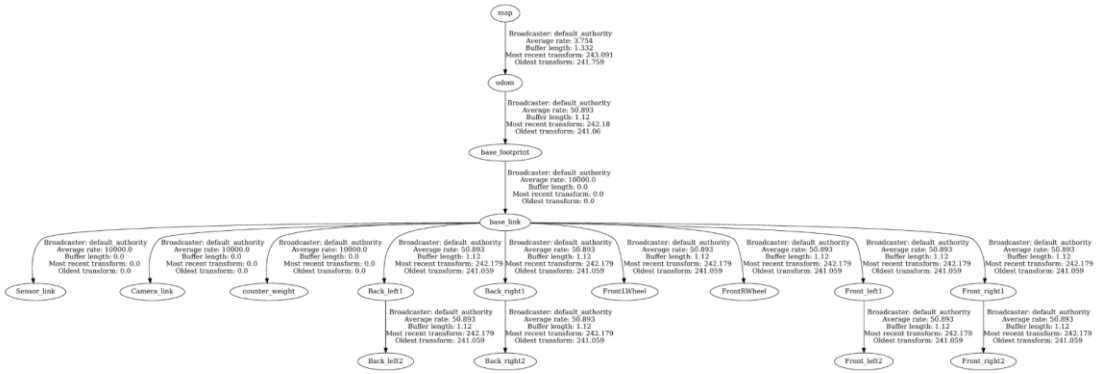

Figure 6: The resulting TF tree with diffdrive wheels, map and odom

4.4 Fundamental structure.

The full SLAM operation within our robot system is structured as a layered pipeline, beginning with simulation and culminating in real-time map generation and localisation. This pipeline ensures the robot is equipped with all necessary environmental awareness to support autonomous navigation.

Simulation and Sensor Data Generation

At the foundation, the robot is spawned in the Gazebo simulation environment, where the configuration specified in the robot's URDF file determines its physical and sensor characteristics—including the placement and properties of LiDAR, wheel encoders (for odometry), and joints. These components produce simulated sensor data in the form of Gazebo-native topics such as:

- /scan2 (LaserScan) -gz.msgs.LaserScan
- /joint_states -gz.msgs.Model
- /odom - gz.msgs.Odometry

To interface this simulation with ROS 2, a **bridge node (gz_bridge)** is initialized in the launch file. This bridge translates Gazebo-specific message types to their ROS 2 equivalents (e.g., sensor_msgs/msg/LaserScan, nav_msgs/msg/Odometry), ensuring compatibility with the broader ROS 2 ecosystem.

Once the ROS 2 topics are active, the **slam_toolbox** node—configured in **online asynchronous mode**—subscribes to the key data streams:

- **Laser scans** (e.g., /scan2) - sensor_msgs/msg/LaserScan

- **Odometry** (/odom) - nav_msgs/msg/Odometry
- **Transform data** (/tf) - tf2_msgs/msg/TFMessage

From these inputs, the toolbox produces two crucial outputs:

- A continuously updated **occupancy grid map** published on /map
- A transform broadcast from **map → odom**, providing the pose estimation chain required for Nav2

This structure allows the robot to simultaneously construct a map and localize itself within it in real time.

Using the **online asynchronous (online_async) mode** of slam_toolbox offers significant advantages when operating under constraints such as limited processing power, low Gazebo Real-Time Factor (RTF), and inconsistent simulation topic rates.

Key Benefits:

1. **Asynchronous Processing:** Online_async mode decouples the scan processing from the main thread, allowing the system to handle incoming data without blocking. This is particularly beneficial when the simulation cannot maintain real-time performance, as it prevents delays in the SLAM process.
2. **Robustness to Inconsistent Data Rates:** In scenarios where sensor data arrives at irregular intervals due to simulation lag or computational bottlenecks, the asynchronous mode ensures that each scan is processed as it becomes available, maintaining the integrity of the mapping process.
3. **Efficient Resource Utilization:**By offloading computationally intensive tasks to separate threads, online_async mode makes better use of available CPU resources, which is crucial when operating on hardware with limited processing capabilities.
4. **Improved System Responsiveness:**The separation of concerns in online_async mode allows the robot to continue navigating and responding to new inputs even while previous scans are being processed, enhancing the overall responsiveness of the system[4].

4.5 Launch code explanation.

In this project, the SLAM node launch configuration adopts a similar implementation approach to the Nav2 official TurtleBot3 example. Specifically, we utilize the online_async_launch.py from the slam_toolbox package as secondary launch file, encapsulated through the IncludeLaunchDescription function. The launch_arguments parameter passing mechanism allows flexible configuration of SLAM-related parameters, including the slam_params_file for specifying the path to the SLAM parameter configuration file and use_sim_time set to true to

ensure correct timestamp usage in the simulation environment. This implementation maintains consistency with the SLAM launch configuration in the Nav2 official TurtleBot3 example, primarily reflected in using the same launch file reference method (PythonLaunchDescriptionSource), adopting similar parameter passing mechanisms, and maintaining a clear and maintainable configuration structure. This design not only ensures reliable implementation of the SLAM functionality but also provides convenience for future functional extensions and parameter tuning.

Additionally, several key parameters in our custom mapper_params_online_async.yaml were modified to better accommodate the specific characteristics of our robot and the practical constraints of running SLAM in a low real-time factor (RTF) simulation environment. The scan_topic and sensor_frame were adjusted to /scan2 and Sensor_link respectively, reflecting the actual topic name and TF frame declared in the robot's URDF for the RPLIDAR sensor. Additionally, the minimum_travel_distance and minimum_travel_heading were reduced from the default 0.5 m and 0.5 rad to 0.1 m and 0.1 rad. These changes were essential for ensuring that the robot—configured as a small differential drive platform—could update its pose graph even during short, frequent micro-movements, which are typical in indoor navigation. The max_laser_range was also shortened from 20.0 m to 8.0 m, which is sufficient for the scale of our simulated arena and helps reduce computational overhead by excluding distant, less-informative scan data. To address the instability introduced by Gazebo's lower RTF and irregular topic publishing rates, we optimized minimum_time_interval from 0.5 s to 0.2 s, allowing more frequent scan insertions, and increased transform_timeout from 0.2 s to 1.0 s to tolerate delayed TF broadcasts. Finally, the queue_size was set to 100 to prevent scan losses due to message congestion during high-load conditions. Collectively, these changes tailor the SLAM pipeline to both the robot's structural setup and the challenges of simulation performance, ensuring stable and responsive mapping and localisation[5].

4.6 Target following code explanation.

The **Gazebo Bounding Box Camera** is a simulated sensor designed to detect and annotate objects within the simulation environment by generating bounding boxes around them. This sensor can be configured to output either 2D or 3D bounding boxes, depending on the <box_type> parameter. For instance, setting <box_type> to 2d enables the sensor to produce 2D axis-aligned bounding boxes around visible parts of objects. The sensor's configuration includes parameters such as <horizontal_fov> for the field of view, <image> dimensions for the resolution, and <clip> planes to define the near and far limits of detection. The <update_rate> determines how frequently the sensor publishes data, and <visualize> enables or disables the visualization of the sensor's output in the Gazebo GUI. The sensor publishes its data to a specified <topic>, allowing other components in the simulation to subscribe and process the bounding box information

The BoundingBoxController node define by BoundingBox_callback.py implements a vision-based control system that generates velocity commands for robot navigation based on object detection results. The node subscribes to the /camera topic, which publishes Detection2DArray messages containing bounding box information of detected objects in the camera view. The control strategy is implemented through the following key mechanisms:

The node processes the vision_msgs/msg/Detection2DArray message to extract the target object's position and size information in camera frame, specifically focusing on the bounding box's center coordinates (center_x, center_y) and width (width_x). The control algorithm calculates appropriate linear and angular velocities based on two main factors: the horizontal offset of the target from the image center and the estimated distance to the target. The horizontal offset is normalized to a range of [-1, 1] relative to the image width, where negative values indicate the target is on the left side of the image, and positive values indicate it's on the right. The angular velocity is directly proportional to this normalized offset, with a negative angular velocity causing the robot to turn right and a positive value causing it to turn left. The linear velocity is determined by the estimated distance to the target, calculated using the ratio of the bounding box width to the image width. When the target is far away (small bounding box), the robot moves faster, and as it approaches the target (larger bounding box), the speed decreases. Additionally, the node implements a safety feature that reduces linear velocity when the target is significantly off-center (normalized_x_offset ≤ -0.5 or ≥ 0.5), ensuring smoother navigation. The calculated velocity commands are published to the /cmd_vel_target topic as Twist messages, enabling the robot to track and approach the target object while maintaining appropriate speed and turning rates.

It's important to note that, similar to how we transitioned our robot's configuration from legs to wheels without affecting the overall navigation stack, the bounding box camera sensor can be replaced with conventional OpenCV-based object detection methods. As long as the OpenCV implementation publishes messages of type vision_msgs/msg/Detection2DArray and provides the center coordinates and estimated width of the target object in the camera frame, the rest of the navigation system can remain unchanged. This modularity ensures flexibility in choosing the most appropriate object detection method based on the specific requirements and constraints of the project[6][7].

4.7 Discussion.

In terms of SLAM performance, the robot was able to achieve high-precision localisation and generate a reliable occupancy grid map while exploring and navigating toward the target. In Scenario 1 and Scenario 2, which involved simplified models and low-complexity structures, the generated maps were accurate, clearly segmented, and updated consistently in real time. This

demonstrated the effectiveness of our modified SLAM parameters and the online_async configuration in maintaining stability and responsiveness under controlled simulation conditions.

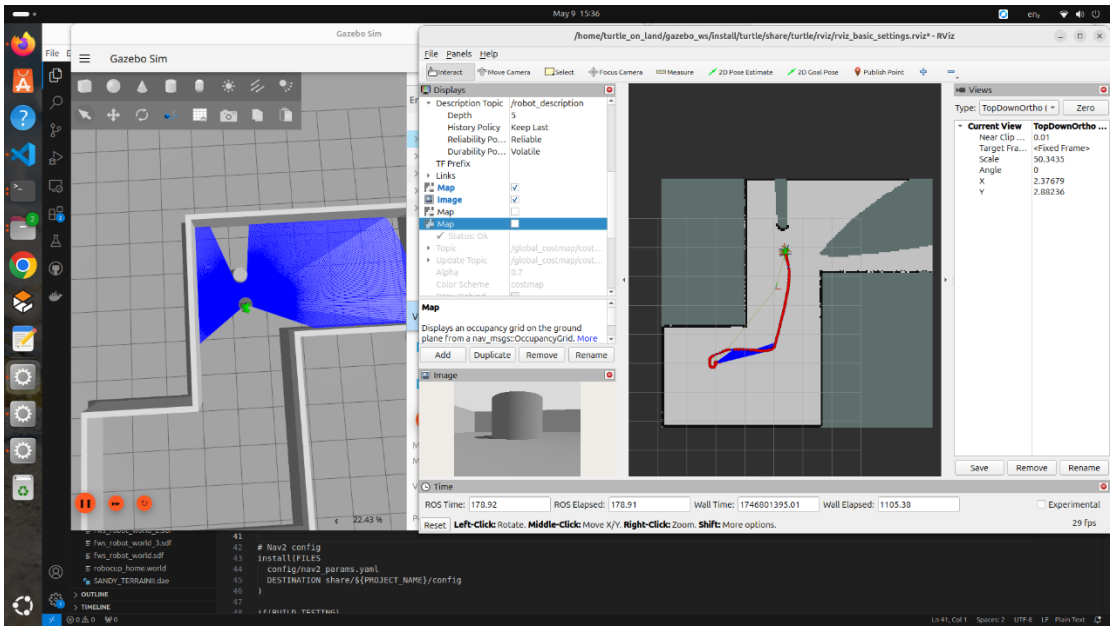

Figure 7: Scenario 1 Slam mapping result with localized trajectory

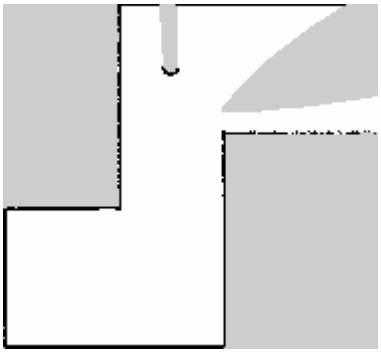

Figure 8: Scenario 1 occupancy grid map

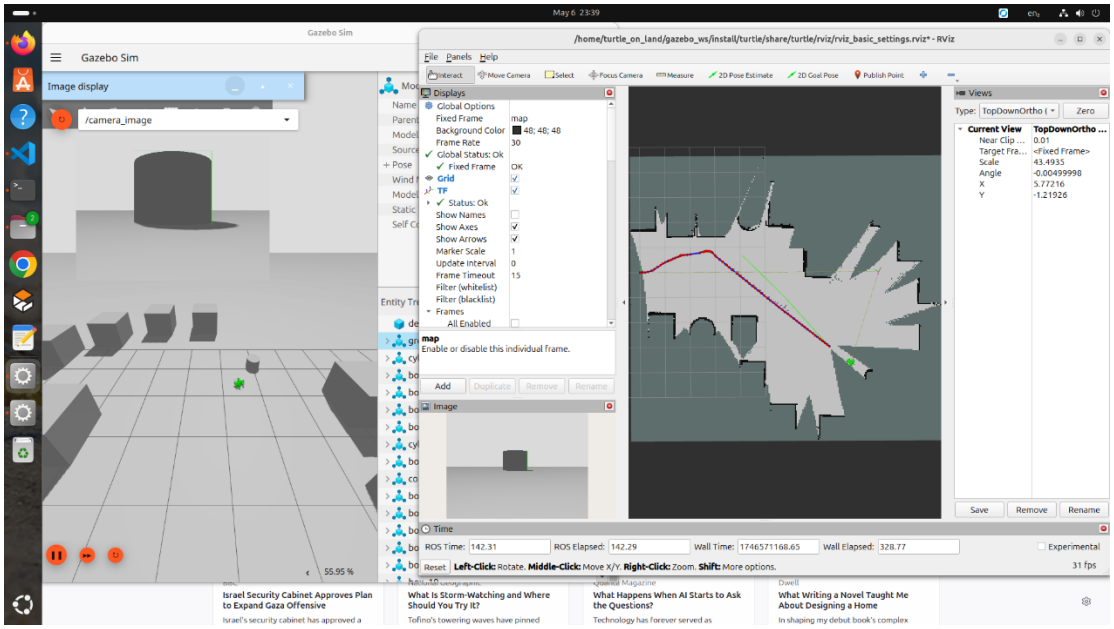

Figure 9: Scenario 2 Slam mapping result with localized trajectory

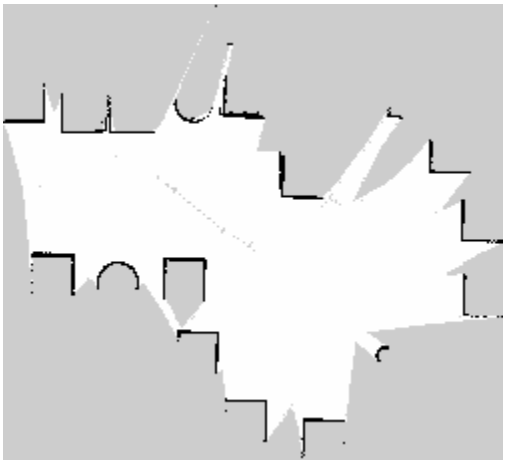

Figure 10: Scenario 2 occupancy grid map

However, in Scenario 3 and Scenario 4, where the robot operated in Blender-generated .dae environments with more complex geometries and larger model meshes, we observed a noticeable degradation in mapping quality. The simulation experienced a significant drop in Real-Time Factor (RTF) as shown in the Figures, while all four CPU cores are at 100% load, which in turn disrupted the continuous publication of the map → odom transform. This instability led to slower map updates, degraded localisation accuracy, and lower-resolution mapping outputs. As a result, the occupancy grid became less useful for path planning, and the robot's performance in these scenarios was notably diminished.

These findings highlight the critical influence of simulation performance on the fidelity of SLAM and navigation. Although the system design, including the fallback bounding box detection and modular ROS 2 architecture, remained functionally intact, the timing inconsistencies induced by low RTF exposed the limits of real-time operation within resource-constrained environments.

Future improvements could include further tuning of SLAM timing parameters, exploring lighter environment models, or deploying on more performant hardware to mitigate real-time bottlenecks. Nonetheless, the system successfully fulfilled its primary objectives, showcasing autonomous navigation and target interaction in multiple test scenarios while offering a robust and extensible framework for future development.

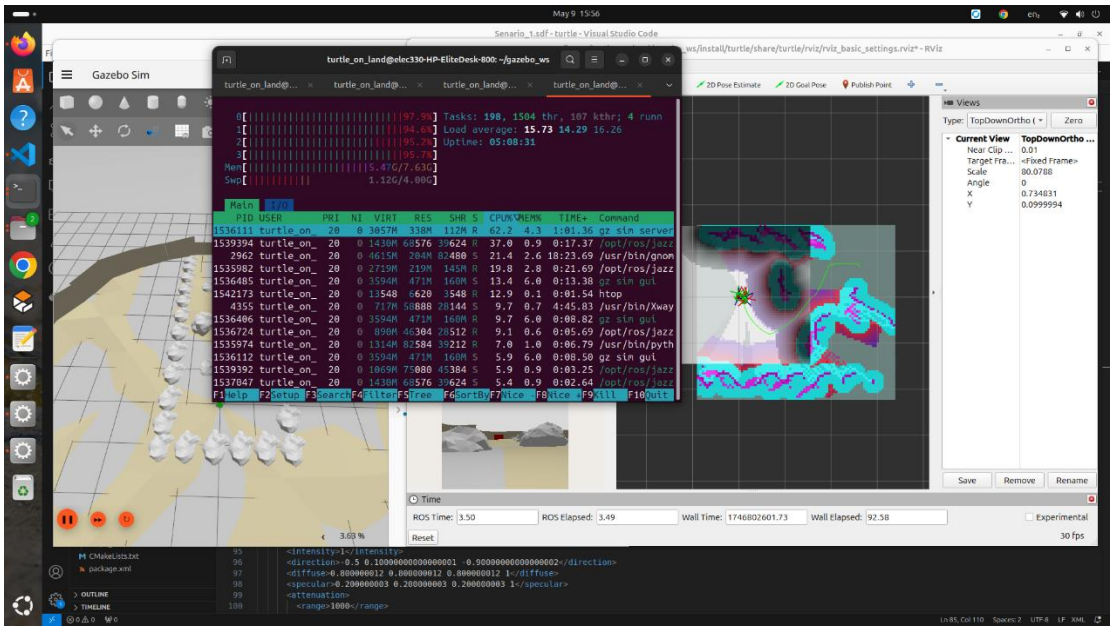

Figure 11: Scenario 3 Slam mapping result with localized trajectory

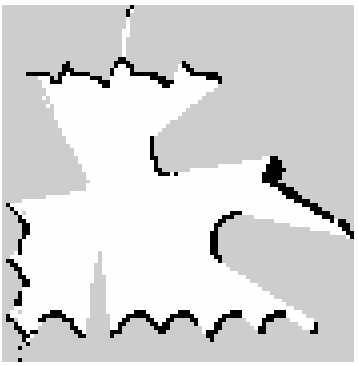

Figure 12: Scenario 3 occupancy grid map

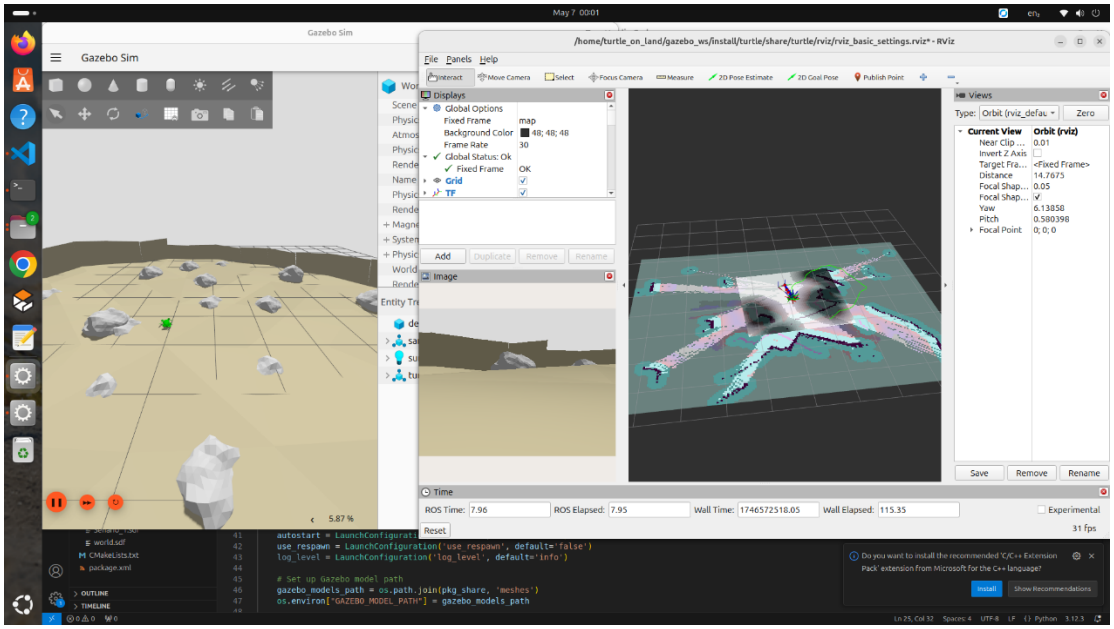

Figure 12: Scenario 4 Slam mapping result with localized trajectory

Figure 13: Scenario 4 occupancy grid map

4.8 Reflection.

The iterative development process provided valuable hands-on experience with robot–simulation interfacing, particularly in configuring and deploying Gazebo plugins such as DiffDrive, and bridging sensor topics into ROS 2. Transitioning from 3D to 2D SLAM gave us the opportunity to explore and compare different SLAM strategies, from lidarslam to slam_toolbox, deepening our understanding of ROS TF architecture and data flow. Furthermore, tuning parameters under

constraints like low processing power and reduced Gazebo Real-Time Factor (RTF) highlighted the practical impact of timing, queue sizes, and sensor configurations on system stability and localisation accuracy. These insights are highly transferable to real-world robotic deployments and have enhanced our systems-level thinking.

In retrospect, a limitation in our early approach was the underestimation of the complexity involved in inverse kinematics for the original legged design. With limited prior knowledge, we did not fully consider how to model joint motions in a way that could reliably yield predictable end-effector trajectories suitable for navigation. More time spent early on clarifying the kinematic feasibility could have improved the initial model's practicality or justified an earlier switch to a wheeled configuration. Nonetheless, the challenge prompted meaningful exploration into motion modelling and SLAM integration, contributing significantly to our learning.

5. Target Detection.

There were two attempted methods for image processing for this project. The first one will be called “Array Analysis” and the second will be called “Image Analysis”. The following paragraphs will detail the theory, implementation, limitations, and complications of both. The “Image Analysis” is inspired and partially comprised of work completed by Calvin Hargis for an aeroponics thesis at the University Of Liverpool, and so this will be quoted and referenced throughout[8].

5.1 Array Analysis.

The Array Analysis method was the first implemented and it revolves around manipulating and processing pixel coordinate data to properly identify the target object. The initial design was outlined in the Assignment 2 paper in more detail[9]. This method comprises seven steps.

1. Receive Image: The target detection algorithm subscribes to the camera topic to receive live video data from the robot.
2. Colour detect: In this step, the image taken is evaluated by colour to isolate the red parts. Using OpenCV, the image is compared to a range of colours as defined by an upper and lower bound of the RGB colour type. This creates a resultant image that is only the pixels that fell in range, with all others being black.
3. Pixel Locating: From the resultant image, the location of each pixel that is not black is saved into a large array. Therefore, every pixel that fell within the colour bounds is now located.
4. Thresholding: The pixels’ individual x and y values are arranged highest to lowest and all that do not fit within the lower 10% or upper 90% confidence threshold (assuming standard distribution) are eliminated. These are then recombined to create a filtered list of pixel coordinates. This step is important to disclude any stray pixels that happen to be within the colour bounds.
5. Centroid Pixel: The pixel coordinates are compared to find the one which lays at the centre of the array, effectively finding the median. This step allows for the algorithm to establish a central point of the target in order to compare it to the overall boundaries of the image.
6. Comparing: The centroid x value is compared to half the width of the image, the central horizontal point. If the x value is higher, then the centroid is to the right of the image centre. If the x value is lower, then it is to the right.

7. LR Decision: The left or right decision from the previous step is sent to the navigation algorithm, which will allow the robot to home in on the target.

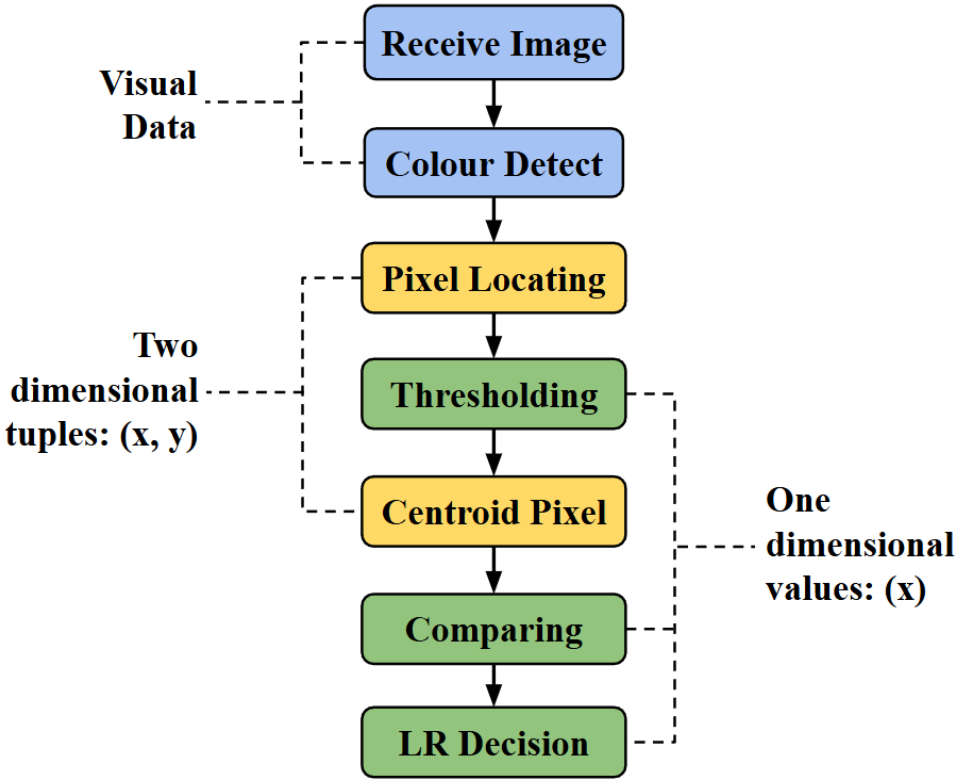

Figure 1: Array analysis diagram.

The main benefit of this method is that it is easily implemented by someone without extensive knowledge of image processing. Apart from the colour detection, the rest of the operation are standard mathematics operations. Any person experienced in python would understand how to apply those operations to various array components. Therefore, it is accessible to a wider potential user base.

The main drawback of this method is the computational cost and the many data types. As can be seen in figure 14, data types are changed four times, from image to tuples, tuples to vector, vector back to tuples, and then tuples again to vector. This made it significantly difficult to edit and change during the development process as a change of data type in one step would significantly affect the others. This method also had so many computations, at times working with an array of more than 18,000 pixel locations, that it crashed the program upon the red target

entering the camera. Attempts were made to trim the number of pixels used, but to no avail. It worked with a static image as shown in assignment 2[9] but wasn't able to perform the same with a video.

This method was ultimately too delicate and large to be finished, as it got slowly bogged down by compounding errors. There was a point in which a live feed of colour isolated camera data was produced, however this was lost in the development process and so is not present in this paper. A new method was created which utilises more image oriented analysis techniques and focuses on data type continuity.

5.2 Image Analysis.

The Array Analysis method was the second implemented and it revolves around using image data and manipulation from the OpenCV library before finally transforming into coordinate data. The work and initial design was created by Calvin Hargis, as previously mentioned, so more information can be found in his thesis[8]. This method comprises eight steps.

1. Receive Image: The target detection algorithm subscribes to the camera topic to receive live video data from the robot.
2. Colour detect: In this step, the image taken is evaluated by colour to isolate the red parts. Using OpenCV, the image is compared to a range of colours as defined by an upper and lower bound of the RGB colour type. This creates a resultant image that is only the pixels that fell in range, with all others being black[8].
3. Grayscale & Blurring: "This step just transforms all the pixels detected in the previous step into gray and then blurs them. This is a standard step for image processing and removes unnecessary colour data. The gaussian blurring makes the edges of images easier to detect for other image processing operations. Gaussian blurring uses a kernel, or pixel matrix, to which adjusts the value of a pixel's neighbours to be more similar to the target pixel, thus blurring the image together. This program uses a rather large 8x8 kernel, due to the fragmented pixel groups scattered across the image."[8]
4. Closing: "The next step, segmentation, requires that the body of the plant be one continuous bunch of not black pixels. To do this, the image is closed using an OpenCV tool with a specified mask, i.e. filter. Closing, in simple terms, is a morphological operation that closes holes and gaps in pixel maps without increasing their overall size. It comprises two smaller morphological operations, dilation and erosion. As demonstrated in figure 15, erosion thins out existing pixels, even erasing features of the shape. Dilation expands on existing pixels, enlarging the shape. Closing is a dilation followed by an erosion, which acts

to fill in the holes on the left side of the original shape a, while preserving the two legs coming out of the right side."[8]

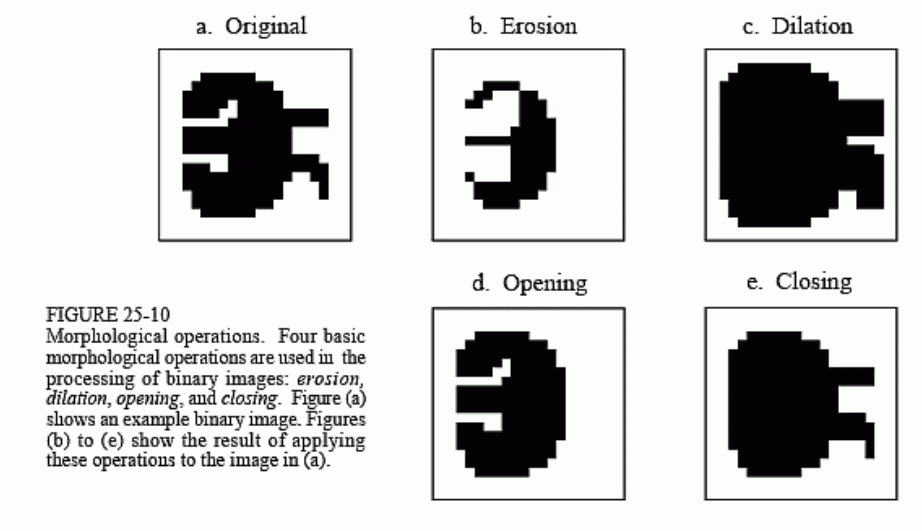

Figure 15: Example of closing[10].

5. Segmentation: "The final image processing step is to eliminate all pixels that may possibly have been accidentally included. Assuming a pixel group was in the right colour range it may have reached this stage and will impact the array analysis section, so it must be removed. The program scans through the entire image and creates another image that only includes contiguous pixel groups larger than 8,000 pixels. This ensures that when the pixel values are scanned, in the next step, only those in the plant are included."[8]
6. Centroid Pixel: The pixel coordinates are compared to find the one which lays at the centre of the array, effectively finding the median. This step allows for the algorithm to establish a central point of the target in order to compare it to the overall boundaries of the image.
7. Comparing: The centroid x value is compared to half the width of the image, the central horizontal point. If the x value is higher, then the centroid is to the right of the image centre. If the x value is lower, then it is to the right.
8. LR Decision: The left or right decision from the previous step is sent to the navigation algorithm, which will allow the robot to home in on the target.

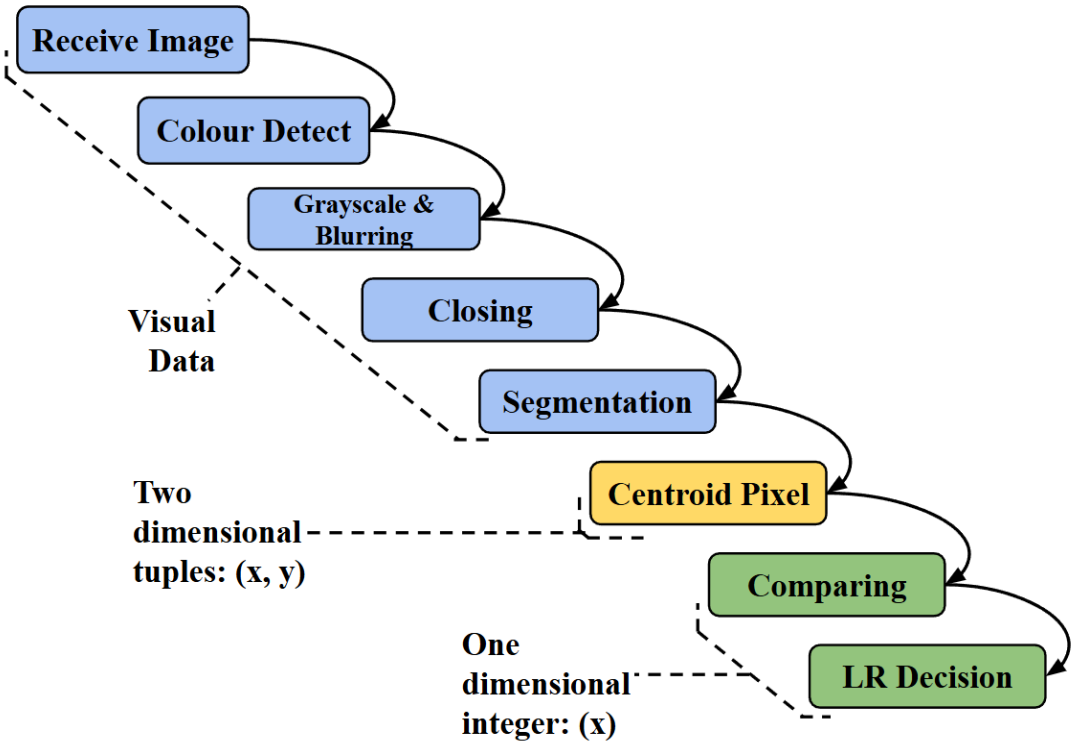

Figure 16: Image analysis diagram.

The main benefit of this method is its simplicity. As can been seen in figure 16, this method only has two switches in data types. This allows for the various image processing techniques to be edited and adjusted without fear that it will negatively impact the functioning of later parts. Furthermore, these steps are much shorter than the previous method, only taking a few lines of code each thanks to the OpenCV library.

The main limitation of this method was time. It was assumed that it would be more time efficient to keep attempting to upgrade the first method, Array Analysis, rather than develop a new one. However, as it became increasingly obvious that the compounding errors of that method were too strong of a barrier to finishing, it was abandoned. The second method only took a few weeks to develop, much faster than the previous, but it was just too close to the deadline to complete. If more time was allowed, there's no doubt that the Image Analysis method would be sufficient to complete the task.

5.3 Results Compared.

To fairly assess the success of this section of the project, a comparison between the two methods is needed. While the method structure and limitations have already been discussed, this section will provide visual evidence to compliment that commentary.

5.3.1 Array Analysis.

This evidence, shown in figures 17 and 18, is drawn from the assignment 2 specification written by Hargis et al. This is because the method was not able to be implemented into video format, so the still images will have to do.

Figure 17: Array analysis image test swab[9].

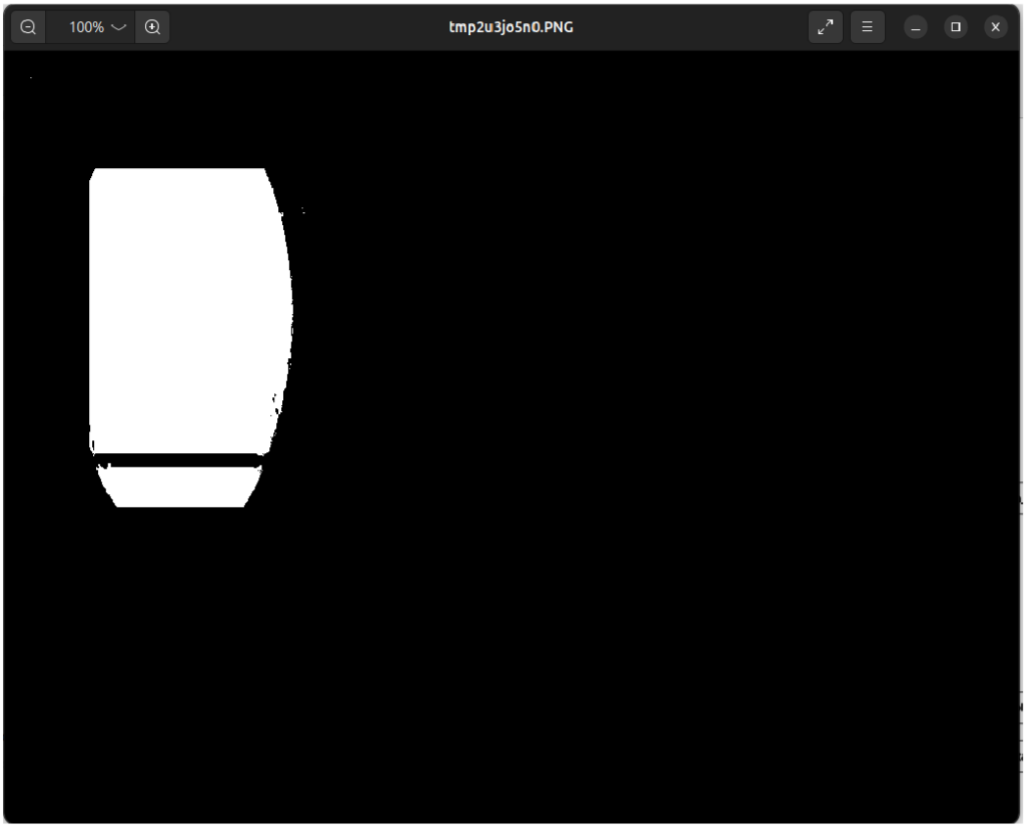

Figure 18: Output from Array Analysis[9].

Figure 17 shows the initial test image for the array analysis method as well as the colour detection result. Almost all of the pixel range was isolated successfully and only a comparatively small amount of stray brown pixels from the rest of the image were kept in. Figure 18 shows the result of the filtering up to the thresholding step, as beyond that no visual data is present. It can be seen that while there is a band of missing pixels, that is because this is from a slightly different colour bound test, figures 17 and 18 are not from the exact same test.

The final output seems to have preserved the shape and position of the target well. The majority of pixels are accounted for, the stray pixels are eliminated, and no unintentional pixels have been added. There are areas of sheering, though, where the target seems to have been sliced off in the top and left sides. This is likely due to an error in the thresholding step, where the pixels are first arranged by magnitude in x and y before being evaluated. It is possible that some were unintentionally eliminated from the body of the target.

5.3.2 Image Analysis.

While not able to be tested on video, the thesis by Calvin Hargis does give some examples of the outcomes relative to his aeroponics goals.

Figure 19: Initial test photo for image analysis[11].

Figure 20: Finial photo for image analysis[8].

Figure 19 shows the test image that Hargis used to test the method originally used for detecting plant matter. Figure 20 shows the final visual output of the method. Clearly there are some holes in the final image. Hargis explains these are because the central parts of the leaves are actually white, a composite of all RGB values, meaning they get filtered out during the colour detection phase. Regardless, the outer bounds of the plant are intact, meaning the centroid should still be reliable.

5.3.3 Comparison.

To compare the two there is clearly one major difference, it is the degree to which pixels are eliminated from the image. While the image analysis method has more holes in the centre, these are widely distributed and do not skew the overall centre of pixel mass that the centroid would be derived from. These holes are also created by white spots, which are not present in the test swab for the array analysis image. This suggests that these are not necessarily comparable and a comparative study using a white-spotted target would be in order.

The array analysis result is missing pixels too, in more smooth and regular patterns. Since they are mostly missing from the top and left side, this may contribute to actually changing the overall centroid that will eventually be found. However, this may only be so obvious because of the regular oval shape of the test swab. By comparison, the irregular pattern of the plant leaves are much easier to gather a comparative sample from.

Despite the differences in testing material, it is clear that the Image Analysis method is much better suited to this task. It is not only much faster, simpler, smaller, and easier to edit, the result is more useful. Since the missing pixels are mostly in the interior and randomly spread throughout, it is less likely to change the centroid as compared to the array analysis method.

If this project was given more time, the image analysis method would be further developed and eventually implemented. However, of use would also be a comparative study of the two methods with similar testing conditions. While the array analysis method is more cumbersome and computationally expensive, it is possible that the precise control offered would be useful to the task at hand. Therefore, a test is needed before this method can definitely be abandoned.

5.4 Discussion.

The current system uses the bounding box camera package[12] to detect the target, as mentioned in section 4.6. While this is a form of image recognition, it relies on pre-tagged objects which use the native ROS2 and Gazebo framework in the URDF files for the simulation. In this way, the robot can navigate to the target that it has already identified as important. This bounding box method, in keeping with the idea that this robot could be used for ecological maintenance, could model a pre-tagged turtle that is getting checked on. Shelly could therefore have a two-layered detection system capable of checking already tagged turtles and cataloguing new specimens.

6. Development of a Simulated Beach Environment with Sand Physics.

As part of the simulated environment created to evaluate a robotic turtle's ability to traverse natural terrain, Shahzeb created a complex virtual beach environment by combining two complementary approaches. Shahzeb used Blender to create the environment's static elements, including the terrain itself, various rock formations, and vegetation, and Gazebo to model the sand dynamics and the robot's physical interaction with the environment. Dividing the work in this way allowed for a visually high-fidelity environment that still maintained a realistic physical response underfoot.

There were four different environments that were developed, each of which served a specific testing objective. One served as the general beach environment, and the other three were more specialised test environments, each of which was designed to challenge various aspects of the robot's navigation and perception systems.

6.1 Overview of the Four Simulation Environments.

Each of the environments were designed with a particular aim and were used to experiment with different capabilities of the robotic turtle, including its locomotion, navigation accuracy, and environmental awareness. Although all four of the environments had the same basic sand physics, they differed significantly in layout and the challenges they presented.

6.1.1 Main Beach Environment.

This was the most comprehensive and visually complete environment. It consisted of the following features:

- An extensive, shaped stretch of beach terrain that mimicked natural seashore contours
- A collection of trees and rocks that were placed in a way that mimicked the way that they would occur in a real coastal ecosystem.
- A clearly defined target location placed at one end of the terrain

This environment was used primarily to validate the robot's general locomotion capabilities, assessing its performance on long-distance travel, and carry out preliminary perception experiments. It also served as a visually rich and diverse space for adjusting and experimenting with various robot parameters in a controlled setup.

6.1.2 Target Navigation Test World.

The second environment was constructed with a much simpler structure to allow for easy observation of the robot's basic navigation capabilities. Its main characteristics were:

- A fairly level terrain surface with only minor sculpting to provide small elevation changes
- Very few obstacles, to allow the robot to move about the space without unnecessary interference
- A fixed goal point located across the environment to be the robot's target

In this world, the robot was required to navigate from its starting position to the goal using its internal localisation and path-planning systems. The purpose of this scenario was performed to test the fundamental navigation pipeline and to monitor speed, directionality accuracy, and stability as the robot moved on the sand.

6.1.3 Obstacle Course with Rocks.

The third world presented a much more physically demanding challenge. It was designed as a narrow path filled with carefully placed obstacles. It featured:

- Rocks of various shapes and sizes arranged in such a way that required accurate movement.
- Narrow passageways that required accurate turning, careful repositioning, and fine motor control of the robot

The robot had to move towards a visible target while it was actively avoiding obstacles in real time. This setup was ideal for testing the robot's capability to:

- Adapt its path when coming across dynamic environmental constraints
- Recalculate routes to avoid collisions

6.1.4 Dense Forested Terrain with Rocks and Trees.

The fourth and final environment was the most complex in structure and ground features. It was modelled to simulate a semi-random outdoor maze and included the following elements:

- A dense concentration of trees and rocks to create a heavily populated terrain
- Uneven terrain surfaces and multiple changes in elevation to simulate the unpredictable nature of real-world outdoor settings
- A variety of possible routes to the goal, some of which were designed to be intentionally ambiguous or difficult to resolve

This challenging setup was used to assess the following aspects of the robot's performance:

- The effectiveness of its sensors in accurately perceiving and differentiating between different environmental objects such as trees and boulders
- The overall robustness of the navigation system when operating in visual obstruction-rich environments with demanding decision-making requirements
- The ability of the robot to evaluate which routes were possible and which were not because of obstacles or slopes

This world effectively mimicked many of the issues that arise in real-world outdoor robotics, such as noisy sensor data, irregular terrain, and unclear routing decisions.

This world was able to replicate many of the difficulties that actual turtles experience on land, due to the changing conditions on coastal areas.

6.2 Environment Modelling in Blender.

The visual and structural aspects of the simulation were constructed using Blender, a feature-rich open-source three-dimensional modelling package. Modelling was intended to create a naturalistic beach setting with sufficient structural complexity to deliver both aesthetic realism and practical functionality within the simulation.

6.2.1 Terrain Sculpting and Optimisation.

To create the beach environment, Shahzeb began by adding a large flat plane to act as the ground surface. Shahzeb then entered Sculpt Mode, where a combination of tools such as the Grab, Draw, and Smooth brushes were used to form gentle slopes, raised dune-like protrusions, and subtle height differences. These features were meant to mimic the different kind of small elevations that are typically found in coastal regions.

To achieve sculpting detail in major areas without unrealistically making the model more complicated, Shahzeb enabled Dynamic Topology (Dyntopo). This allowed Blender to subdivide the mesh itself wherever additional detail was required. Once sculpting was completed, a Subdivision Surface Modifier and a Multiresolution Modifier were applied to further refine the surface and enable smoother transitions of shading over the terrain.

To maintain compatibility with Gazebo whilst enhancing performance, Shahzeb minimised the overall quantity of polygons. The Decimate Modifier was utilised to reduce the quantity of faces where they were unnecessary but retained detail only where it was visually or functionally relevant.

6.2.2 Environmental Props: Trees, Rocks, and Debris.

To generate realism and offer natural navigation challenges for the robot, Shahzeb populated the environment with a variety of natural features. This included:

- **Rocks**, which were modelled from non-uniform base meshes. The Displace Modifier and bespoke procedural textures were applied to generate rugged, jagged shapes that mimicked natural rock forms.
- **Trees**, which were produced using Blender's Sapling Tree Gen add-on. The add-on provided precise control over trunk height, curvature, and leaf density. In preparation for simulation, the trees were down-sized by removing high-polygon leaf meshes and limiting the number of subdivisions in the branches.

Each of the objects were textured with a combination of image-based and procedural materials. Rocks, for instance, were assigned albedo maps that replicated stone texture, and trees received bark normal maps. Although Gazebo is not yet completely compatible with Blender's native material system, most visual detail was preserved for Blender-based rendering and was preserved to a degree in Gazebo through the use of mapped image textures.

6.2.3 Boundary Walls: Justification and Implementation in the Simulated Sandy Terrain.

Along the edges of the beach, boundary walls were included in the design of the test environment for testing robotic TurtleBot on Gazebo. They were not simply created for aesthetic reasons as they had to be used to ensure the simulation is not only functionally stable but also avoid the chance of the turtle falling off the edge of the map.

6.2.4 Rationale for Including Boundary Walls.

The primary reason for including boundary walls was to prevent the robot from wandering off the boundaries of the terrain mesh and into unknown space. Without physical boundaries in physics-based simulators such as Gazebo, there can be numerous problems such as:

- The robot can tumble off the edge of the terrain into an undefined three-dimensional void and get trapped there
- The simulation will become error-prone or unstable with the robot assuming unsupported or incorrect positions
- Sensor measurements will be inconsistent or unreliable, severely affecting SLAM (Simultaneous Localisation and Mapping) performance
- Test sessions will be interrupted on a regular basis due to unplanned or unrealistic robot behaviour.

By enclosing the terrain with walls, the simulation was made more realistic of real world set ups, such as open air sandpits, roped arenas, or enclosed test fields. These boundaries created a predictable area within which the robot could operate, supporting:

- Greater consistency in data obtained from repeated test runs

- More accurate in comparing algorithmic performance
- A smoother debugging and refinement process for navigation strategies

6.2.5 Technical Implementation in Blender.

The boundary walls were created directly within Blender using its mesh editing tools. After sculpting and optimising the beach ground, Shahzeb used Edit Mode in order to define the area where the boundary walls would be constructed.

The process involved selecting the outer edges of the terrain mesh and extruding them up vertically using the Extrude feature. The extrusion was only on the Z-axis so that it would be properly aligned with Gazebo's gravity model.

To make the walls structurally realistic and physically functional, Shahzeb applied a Solidify Modifier to provide them with thickness. The final dimensions of the walls were chosen to be similar to real-world equivalents, such as low retaining walls or the wooden edges of a large sandbox. This added physical realism, enhanced collision detection, and reduced the incidence of simulation artefacts such as mesh pass-through.

6.3 Functional Contribution within the Simulation.

Apart from their basic role of containment, the boundary walls had a significant role to play in the interaction of the robot with the world. Through their presence, the robot was able to carry out the following basic functions:

- **Path planning**, where the robot had to take into account non-navigable space and adjust its paths accordingly
- **Obstacle avoidance**, since the walls were immovable obstacles that the robot's sensors would have to sense and evade
- **SLAM and mapping**, as enclosed environments have more stable loop closures and spatial accuracy

From a human perspective, walls also reduced the need for monitoring. As the robot is now in a defined environment, testers could execute autonomous tests without the concern of the robot going out of screen or into invalid space. Walls also made the simulation feel more professional, offering:

- A more professional appearance for demonstrations or presentations
- A better reflection of real-world constraints, making the testing environment more valid
- Clearer communication of environmental structure when discussing results with lectures and other peers

Scene Assembly and Export

In this project, Shahzeb tried to simulate a mobile robotic system navigating through a custom outdoors environment using ROS 2 and Gazebo. To make the testing arena for the robot's path planning and localisation algorithms more authentic, Shahzeb modelled a sandy terrain simulation within Blender. Although Blender provides advanced tools for creating detailed three-dimensional assets, integrating these in Gazebo involved a number of technical challenges. These issues arose from the incompatibility of file formats, coordinate systems, and the requirements of the simulation environment.

Exporting from Blender

The first major obstacle in the integration process was exporting the terrain out of Blender in a form that Gazebo would accept and utilise effectively. Blender supports a range of file types, but the format most compatible with Gazebo is COLLADA (with a .dae extension), as it preserves both mesh and geometry data.

However, exporting to this format was not entirely straightforward. Several technical problems arose during the process:

- The normals on some of the mesh faces had been flipped, resulting in visual anomalies and unexpected collision behaviour in Gazebo.
- Blender uses a Z-up coordinate system, whereas Gazebo works with a Y-up frame of reference. This difference required re-orientation of exported assets manually.
- The difference in unit scales from Blender and Gazebo led to issues of object size at times. Unless adjustments were done slowly and carefully, the exported terrain would appear either too small or abnormally large.
- Materials and textures rarely exported correctly because Gazebo is incapable of supporting Blender's more advanced rendering engines, such as Eevee or Cycles.

To work around these limitations, Shahzeb exported terrain as a COLLADA file, and afterwards manually tuned the model to prepare it for Gazebo. This involved making sure that mesh normals, proper transformations, and asset simplification were conducted in a way to maintain real-time simulation performance.

Creating a Custom Model for Gazebo

Once the terrain had been exported, Shahzeb had to set it up as an accepted model in Gazebo's directory hierarchy. This meant creating a specific folder within the Gazebo models directory, commonly located at ~/.gazebo/models/. In this folder, Shahzeb copied the following files:

- The exported .dae model file(e.g. sandy_terrain.dae).
- A model.config file, which provided metadata such as the model's name, the version number, and the author information.

- A model.sdf file, which defined the visual and physical properties of the model in Gazebo. This file utilised the COLLADA mesh for appearance and collision.
- The texture pictures for the rocks, which allowed for the textures taken from Blender to be shown in Gazebo when the simulation was ran

Shahzeb created a Gazebo world file called sandy_terrain.world after making sure the model was properly structured.

- A <include> element to load the custom terrain model was part of this world file.

- The lighting and ground plane that Gazebo uses by default to create a scene that makes sense visually.

- To avoid accidental collisions upon initialization, a TurtleBot3 robot model is positioned slightly above the surface.

Shahzeb kept this world file in a specific directory for better organisation, making it simple to find and edit as needed.

6.3.1 Launching the Simulation.

After setting up everything, Shahzeb used the terminal to launch the simulation by typing the following command:

gz sim/home/sandy_world.world/gazebo_worlds/turtle_on_land

Gazebo loaded the world successfully, and the terrain appeared as expected. At spawn, the TurtleBot3 was positioned correctly and stayed steady. Shahzeb observed that the robot reacted correctly to environmental boundaries, surface resistance, and collisions with terrain features.

Shahzeb created a Python launch file in order to completely incorporate this simulation into the ROS 2 workflow. This script was in charge of:

- Launching the Gazebo simulator with the customized world.

The substantial disparity between design and simulation platforms was brought to light in this section of the project. Gazebo requires a rigid structure and simplified assets to function effectively, whereas Blender is excellent at producing visually detailed models. Careful export settings, manual configuration, and the addition of simulation-specific metadata were necessary to bridge the gap between these tools.

In spite of the challenges, Shahzeb managed to create a realistic beach environment in Gazebo that provided significant challenges for robotic testing. The finished configuration was a big step toward building a simulation that closely resembles the complexity of the real world and provides a far more trustworthy testbed for assessing robot performance.

Sand Physics Implementation in Gazebo

Though the environment's static geometry and visual detail were designed in Blender, its dynamic physical interaction elements, i.e., the behaviour of the ground which was sandy, were handled by Gazebo. By separating the physical simulation from the visual model, Shahzeb was able to adjust physical realism without sacrificing rendering performance.

Defining the Physical Ground Plane

Shahzeb created a ground plane in Gazebo to symbolise the sandy surface. This was accomplished either by making a native Gazebo plane object or by importing a mesh from Blender. Precisely calibrated surface properties were used to mimic the physical feel of sand rather than simulating individual sand particles, which would be computationally costly and impractical for real-time performance.

The <collision> and <surface> tags of the terrain model's SDF (Simulation Description Format) file contained the pertinent configuration. The way the robot's legs or wheels would engage with the ground was specified by these parameters.

<surface>

<friction>

<ode>

<mu>2.5</mu>

<mu2>2.5</mu2>

</ode>

</friction>

<contact>

<ode>

<kp>1000.0</kp>

<kd>10.0</kd>

<max_vel>0.02</max_vel>

<min_depth>0.001</min_depth>

</ode>

</contact>

</surface>

Following iterative testing and modification, these values were chosen. The grip and resistance normally experienced when moving across dry sand were simulated in part by the high friction coefficients, denoted mu and mu2. As a result, the robot had to work against the surface and was unable to glide as smoothly. The robot was able to partially sink into the ground without experiencing excessive bouncing or instability due to the soft, deformable feel produced by the contact block's stiffness and damping values. The robot appeared to be moving through a loose, granular material thanks to the maximum velocity and minimum depth parameters, which helped to guarantee steady and consistent contact.

Simulating the Feel of Sand

The robot moved realistically across the terrain, despite the lack of granular particles. During the simulation, there was observable resistance from the robot's wheels or feet when it turned or accelerated. The robot's traversal speed was slowed down, and depending on how its weight was distributed and how big its contact points were, it would sometimes partially embed in the ground.

Testing recovery and movement techniques benefited greatly from this physical layer of realism. The simulation appropriately assessed behaviours like reversing, pivoting, or rerouting if the robot got stuck or failed to acquire traction, for example.

6.4 Robot Interaction Testing.

Shahzeb started experimenting with how the robot interacted with the sandy environment after the terrain was completely integrated into Gazebo and the surface physics were set up. To prevent any unanticipated collisions at spawn time, the TurtleBot3 model was loaded into the simulation and placed slightly above the terrain surface.

Shahzeb noticed a few crucial behaviours during these tests that verified the physical parameters were operating as planned:

- The robot's path was impacted by slopes and uneven areas, which occasionally resulted in minor slips or the need to readjust orientation.

- In order to replicate the difficulties faced on loose terrain in the real world, some wheels or parts of the chassis would partially press into the ground when the weight was distributed unevenly.

The significance of precise contact physics was brought to light by these interactions. Neither visually complex surface shaders nor complex particle systems were used in the simulation. Rather, it concentrated on creating a realistic physical response, which made it possible to analyze navigation algorithms, sensor responses, and movement strategies practically.

Visual and Functional Integration

Basic UV-mapped textures can be applied with Gazebo, despite the fact that it lacks many of Blender's more sophisticated rendering features, like procedural shaders and high dynamic range lighting. Shahzeb therefore made sure that all Blender-exported visual models contained correctly unwrapped UV maps and related image textures.

Each object in the SDF files had a <material> tag referencing these textures. Even though the objects did not look exactly like the renders in Blender, they still had enough visual detail to make the simulation both visually appealing and educational.

Shahzeb created a simulation environment that was both physically meaningful and aesthetically pleasing by fusing logical world structuring, Blender visual models, and Gazebo's adjusted surface properties. The robot encountered realistic challenges, and its responses could be meaningfully interpreted to improve performance.

6.5 Discussion.

The development of this simulated beach environment successfully created a functional testbed for evaluating robotic navigation in sandy terrain. By combining Blender's modelling capabilities with Gazebo's physics engine, Shahzeb established four distinct testing environments that effectively assessed different aspects of robotic performance. The main beach environment provided comprehensive testing grounds, while specialised areas like the obstacle course and forested terrain enabled targeted evaluation of specific capabilities.

Key achievements of the project include:

- Creation of visually realistic terrain with natural contours and obstacles
- Implementation of adjustable surface physics to approximate sand behaviour
- Modular design allowing isolated testing of navigation subsystems
- Successful integration between visual modelling and physics simulation

However, several technical constraints impacted the simulation's fidelity. Most significantly, hardware limitations prevented the implementation of true granular sand physics. The available workstation lacked sufficient processing power for:

- Real-time particle simulation
- Dynamic terrain deformation
- Complex granular interactions

This forced Shahzeb to implement simplified friction models that, while functional, could not fully capture the nuances of real sand behaviour. The hardware constraints also caused noticeable performance degradation in more complex environments, particularly those with numerous obstacles or detailed terrain features.

The project revealed several important considerations for simulation-based robotics research:

Performance versus Realism Trade-offs:

Shahzeb had to carefully balance physical accuracy with computational feasibility. While commercial solutions like Webots or NVIDIA Isaac Sim offer more advanced physics, our open-source approach provided greater customisation at lower cost.

Workflow Challenges:
The Blender-to-Gazebo pipeline required numerous manual adjustments for:

- Coordinate system conversions
- Mesh optimisation
- Material/texture translation

Future Development Paths.
To enhance the simulation's capabilities, we recommend:

1. Hardware upgrades to support granular physics
2. Implementation of advanced physics plugins
3. Real-world validation for parameter calibration
4. Cloud-based computation for intensive simulations

This project demonstrates that even with limited resources, valuable robotic testing environments can be developed. The experience underscores how simulation ambitions must be tempered by practical constraints, while still providing meaningful test conditions. The framework established here serves as a solid foundation for future enhancements as computational resources become available.

7. Conclusion.

For Shelly to model an environmental biometric surveillance robot, there are a few key factors that need to be met. One of those is effective navigation. In modelling navigation, this shelly is able to follow a planned path trajectory to a labelled target and park next to it. Using global and local costmaps, the robot can maintain its location relative to its environment and navigate towards the lowest cost path. Thus, such LiDAR based navigation and sensor integration is what drives Shelly's ability to explore her environment. There are some artefacts, errors, and adjustments needed to be addressed, but the reliability for a protytpe is high enough to demonstrate the reconnaisance model outlined in the introduction.

The navigation enables Shelly to operate autonomously, exploring a protected environment and avoiding obstacles. She is not likely to get lost when tracking a known target because of her self-localisation properties and also her ability to recognise the shortest path in front of her. It is possible that the limitations and errors mentioned in section 3.5 may become compounded when operating in a complex real environment. If so, a new round of testing and redesigning would need to follow data gathering to address these issues.

The second critical aspect is mapping. The mapping produced in this project was reliable, accurate, useful, and limited. Several trials, scenarios 1 and 2, showed that outputs could be reliable, useful, and repeated. However, trials 3 and 4 demonstrated the limitations of the current code when faced with more complex environments. This is mainly a hardware limitation, demonstrated by the near 100% CPU use. Seeing as this is also rendering the environment itself, it is possible that only minor upgrades would be necessary to implement this in the real world.

Mapping and navigation are intrinsically linked and the performance of one relies heavily on the other. A reliable and useful map allows a well-designed navigation protocol to function seamlessly. However, if the map is not accurate, then the navigation will be running well but on bad data. Likewise, if the navigation protocol is not developed enough, then a good map may be useless. Ultimately, this is why Kisinguh and Hong worked so closely together on Shelly and were able to produce a developed and servicable prototype. Shelly is currently, with regards to mapping and navigation, a very good demonstration of the kind of autonomous or remotely controlled navigation in threatened environments as outlined in the introduction.

Another key factor is the target detection. While Shelly may be able to navigate to a pre-known target, modelled as an already tagged turtle, a key aspect of the theoretical use case is to detect new specimens. With that in mind, two different types of image processing were created. The Array Analysis model was more robustly able to be controlled, more based in mathematic

principles, and required less experience in image processing to operate. However, this model was also computationally expensive, slow, and ultimately failed to produce a workable outcome. The Image Analysis model is much faster, smaller, and easier to modify, but requires more outside expert knowledge on image processing beyond what is available in ELEC330. Ultimately, time constraints were what stopped this just short of completion.

While both these image processing models have potential, it seems that this project is better suited to the Image Analysis method. As previously stated, however, a stronger comparative study is needed to properly determine this. In terms of the model, Shelly's ability to detect new specimen, either for cataloguing or approaching, is vital and so further work on the Image Analysis method would be undertaken if this project were repeated.

Finally, a key aspect of this assignment is testing, and for that several different worlds were created. The creation of the beach, test, rocks, and forrest worlds were the bedrock for verifying this project's success. As previously stated, the differing environments posed unique challenges, especially the sand and slopes. This showed the team that Shelly was not yet ready to be simulated in an extremely lifelike world. However, there is some evidence, such as the near 100% CPU usage, that this may also stem from rendering the environment itself. Regardless, the worlds are useful as they prove the only use-case upon which we can identify robot problems.

The trials from these worlds demonstrate to the team that there is more preperation needed in robust navigation and mapping before a physical model or final product can be reached. However, they also demonstrate great success so far with the simpler maps. Possibly further code development and more robust hardware, like that laid out in the bill of materials, will boost the performance of the robot and elevate it to the next level of our model.

8. Appendices.

8.1 Appendix 1: ROS2 control.

Although the main robot functionality has been adapted to operate with wheels for its general movement, the individual joints of the robot including the flippers are functional using the ROS2 control framework.

This can be achieved by re enabling the ROS2 control package through the inclusion of a properly configured .yaml file that specifies the required controller, which in this case would be the forward velocity controller. Subsequently, the relevant .xacro file should be reinstated to define the robot joints that will interface with this controller, to initialize the ROS2 control within the gazebo environment, the necessary node can be added to the launch file, which would invoke the controller manager to start both the forward velocity controller and the joint trajectory controller.

To ensure the joints move as the robot drives forward using the differential drive controller, a python subscriber function can be implemented to subscribe to the “cmd_vel” topic and publish the “geometry/twist” message to the forward velocity topic. The added functionality of the robot's joints would allow the robot to transverse rough terrain with less effort.

8.2 Appendix 2: Simulation Time Error.

A common issue encountered during development was the simulation time mismatch error. This typically stemmed from the lack of synchronization between the Gazebo simulation clock and RViz, often cause by not setting the “use_sim”time:true” parameter. This oversight led to various problems across the whole system, and unfortunately due to the current ROS2 node architecture there is no built-in mechanism to automatically identify which nodes are missing this parameter. Figure 21 is a common result of this error where the lidar scan topic is made unavailable to the NAV2 stack as the lidar transform cannot be published to the map frame due to inconsistent clock times.

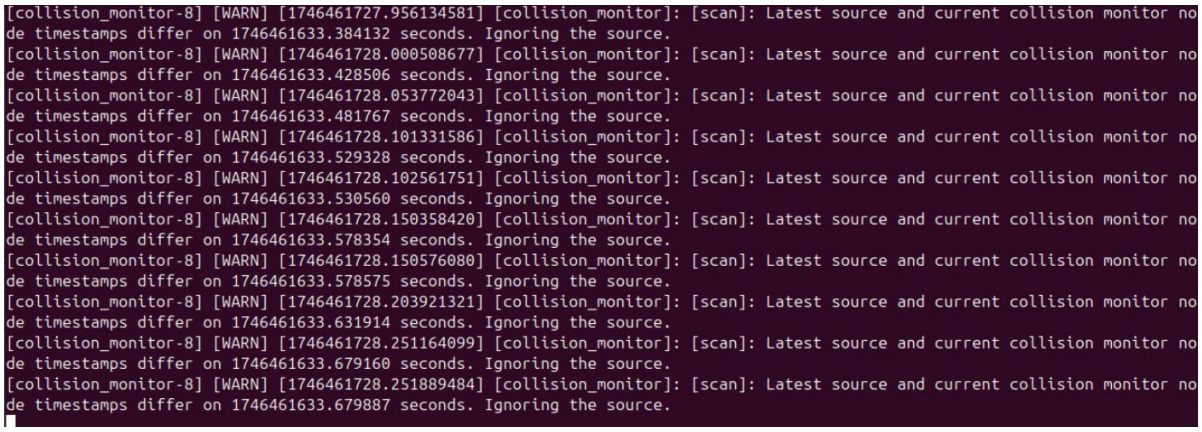

Figure 21. Simulation Time Error

8.3 Appendix 3: Sensor Hardware Implementation.

Integrating physical LiDAR and IMU hardware into the ROS2 system requires adherence to standard conventions defined in the official ROS2 Enhancement Proposals (RESPs) and community best practices. According to REP 105, coordinate frames such as base_link, imu_link , and lidar_link must be defined and connected using the TF2 library to ensure spatial consistency. REP 103 outlines the use of East North-Up (ENU) coordinate system with standard SI units, for IMU integration, REP 145 provides further details specifying that the data should be published using the “sensor_msgs/msg/Imu2” message type with orientation expressed as a quaternation and angular velocity and linear acceleration reported in standard units. Lastly, the LiDAR sensor is to use the “senosr_msgs/msg/LaserScan” message type for 2D LiDAR data .

To integrate the Tyenaza MPU6050 9 axis IMU and the DTOF D300 LiDAR sensor into the ROS2 system, several key modifications are required, the MPU6050 communicates via I^2C and lacks an official ROS2 driver, so integration will involve either writing a custom node using python to read I^2C data and publish it as “sensor_msgs/Imu”, or using a microcontroller to transmit IMU data over a serial interface to ROS2. For the D300 LiDAR which outputs data via UART, integration can leverage existing ROS2 drivers compatible with the LD19 protocol, such as the “ldlidar_stl_ros2”.

8.4 Appendix 4: Diffdrive URDF define.

```
<plugin filename="gz-sim-diff-drive-system" name="gz::sim::systems::DiffDrive">
        <left_joint>
          FLWJoint
        </left_joint>
        <right_joint>
          FRWJoint
        </right_joint>

        <wheel_separation>0.065</wheel_separation>
        <wheel_radius>0.0175</wheel_radius>

        <max_linear_acceleration>1.0</max_linear_acceleration>
        <max_angular_acceleration>2.0</max_angular_acceleration>
        <max_wheel_acceleration>10.0</max_wheel_acceleration>

        <min_velocity>-10.0</min_velocity>
        <max_velocity>10.0</max_velocity>

        <frame_id>odom</frame_id>
        <child_frame_id>base_footprint</child_frame_id>
        <odom_topic>odom</odom_topic>
        <tf_topic>/tf</tf_topic>
        <topic>/cmd_vel</topic>
      </plugin>
```

8.5 Appendix 5: SLAM Node Launch define.

```
online_async_slam = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
```

```
        os.path.join(
            get_package_share_directory('slam_toolbox'),
            'launch',
            'online_async_launch.py'
        )
    ),
    launch_arguments={
        'slam_params_file': mapper_params_online_async_file,
        'use_sim_time': 'true'
    }.items()
)
```

8.6 Appendix 6: SLAM Node launch reference.

```
slam_launch_file = os.path.join(slam_toolbox_dir, 'launch', 'online_sync_launch.py')
declare_params_file_cmd = DeclareLaunchArgument(
    'params_file',
    default_value=os.path.join(bringup_dir, 'params', 'nav2_params.yaml'),
    description='Full path to the ROS2 parameters file to use for all launched nodes',
)
```

```
IncludeLaunchDescription(
            PythonLaunchDescriptionSource(slam_launch_file),
            launch_arguments={'use_sim_time': use_sim_time,
                              'slam_params_file':params_file}.items(),
            condition=IfCondition(has_slam_toolbox_params),
        )
    )
```

8.7 Appendix 7: BoundingBox camera URDF define.

```
<sensor name="boundingbox_camera" type="boundingbox_camera">
        <topic>camera</topic>
        <camera>
          <box_type>2d</box_type>
          <horizontal_fov>1.094</horizontal_fov>
          <image>
            <width>800</width>
```

```
        <height>600</height>
       </image>
       <clip>
        <near>0.1</near>
        <far>10</far>
       </clip>
      </camera>
      <always_on>1</always_on>
      <update_rate>10</update_rate>
      <visualize>true</visualize>
    </sensor>
```

8.8 Appendix 8: References.

[1]H. K Matthew, “ROS2 Navigation framework.” [Online]. Available: https://roscon.ros.org/2019/talks/roscon2019_navigation2_overview_final.pdf

[2] “Utilizability of Navigation2/ROS2 in Highly Automated and Distributed Multi-Robotic Systems for Industrial Facilities,” IFAC-PapersOnLine. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405896322003330?ref=pdf_download&fr=RR-2&rr=9411dac62ffdb3ab

[3] "DiffDrive Class Reference," *Gazebo Sim*, Version 8.9.0. [Online]. Available: https://gazebosim.org/api/sim/8/classgz_1_1sim_1_1systems_1_1DiffDrive.html

[4]"Navigating While Mapping (SLAM)," *Nav2 Documentation*, Open Navigation LLC. [Online]. Available: https://docs.nav2.org/tutorials/docs/navigation2_with_slam.html:contentReference[oaicite:3]{index=3}

[5]S. Macenski, *slam_toolbox: mapper_params_online_async.yaml*, GitHub repository, [Online]. Available: https://github.com/SteveMacenski/slam_toolbox/blob/ros2/config/mapper_params_online_async.yaml

[6]Gazebosim, "Bounding Box Camera," *Gazebo Rendering API Reference*, Version 7.5.0. [Online].

Available: https://gazebosim.org/api/rendering/7/boundingbox_camera.html

[7]Gazebosim, ”Bounding Box Camera in Gazebo," *Gazebo Sensors API Reference*, Version 9.1.0. [Online]. Available: https://gazebosim.org/api/sensors/9/boundingbox_camera.html

[8]: C. Hargis, “Hanging Gardens: Self regulating, self monitoring, aeroponics to model for food-insecurity relief,” University of Liverpool, Apr. 30, 2025.

[9]: C. Hargis, S. Hong, K. Kisinguh, S. Pasha, “Executive Summary for Group 3, Sea Turtle On Land,” University of Liverpool, Dec. 15, 2024.

[10] DPS Guide, “Morphological Image Processing,” *Dspguide.com*, 2024. [Online]. Accessed Mar. 25, 2025. Available: https://www.dspguide.com/ch25/4.htm.

[11] GardeningSG, “Kale,” *Nparks.gov.sg*. Accessed Apr. 25, 2025. [Online] Accessed Apr. 28, 2025. Available: https://gardeningsg.nparks.gov.sg/page-index/edible-plants/kale/.

[12] Gazebo Rendering API Reference “Gazebo Rendering: Bounding Box camera,” *Gazebosim.org*, 2025. [Online] Available: https://gazebosim.org/api/rendering/7/boundingbox_camera.html (accessed May 18, 2025).